

Scene State Machine

menu · play · game-over

one game loop, swap the scene inside it.

Where we left off — Day 11 sound + animation

Yesterday you wired up:

- **pygame.mixer** load .wav files, play SFX on collisions
- **animation index** step through frames at a fixed rate
- **paddle squish** swap surfaces per frame for the hit feel

The catch

Your brick-breaker only knows one state: playing. Press start and you're in. Lose? The screen freezes. Win? Same.

Today: a clean way to model menu, play, paused, game-over as separate objects the game loop swaps between.

The monolithic loop problem

What this page is about: every small game starts with one giant while-loop. As soon as you add a menu or a game-over screen, that loop bloats into spaghetti. Scenes split it apart.

BEFORE — one loop, all states

```
while running:
    if state == "menu":
        draw_menu()
        if start_clicked: state = "play"
    elif state == "play":
        update_paddle(); update_ball()
        check_collisions(); draw_play()
        if lives == 0: state = "over"
    elif state == "over": draw_over()
    # 200 more lines like this...
```

AFTER — one loop, swappable scene

```
while running:
    for event in pygame.event.get():
        manager.handle_event(event)

    manager.update(dt)
    manager.draw(screen)

    pygame.display.flip()
    dt = clock.tick(60)

    # manager holds the active scene.
    # the loop never grows.
```

02 · THE INTERFACE

Scene base class — three methods

What this page is about: a Scene is anything that knows how to respond to events, advance one step in time, and paint itself. Every menu / level / game-over inherits this contract.

```
class Scene:
    """Base class. Subclass for each game state."""

    def __init__(self, manager):
        self.manager = manager

    def handle_event(self, event):
        # called once per pygame event
        pass

    def update(self, dt):
        # advance physics / timers by dt seconds
        pass

    def draw(self, screen):
        # paint to the surface
        pass
```

The three-method contract

handle_event(event)

react to keypress, mouse click, quit. Called once per event in the queue.

update(dt)

advance state by dt seconds. Move balls, tick timers, check collisions.

draw(screen)

render onto the surface. No game logic here — just paint what update produced.

03 · THE ENGINE

SceneManager — holds the active scene

What this page is about: the manager is a thin wrapper. It holds exactly one active scene and forwards events / updates / draws to it. Calling `go_to()` swaps the scene out.

```
class SceneManager:
    def __init__(self):
        self.active = None

    def go_to(self, scene):
        # swap in a fresh scene
        self.active = scene

    def handle_event(self, event):
        self.active.handle_event(event)

    def update(self, dt): self.active.update(dt)
    def draw(self, screen): self.active.draw(screen)
```

Mental model

Game loop talks to the manager, not the scene.

game loop

↓ forwards event / update / draw

SceneManager

↓ .active scene receives the call

MenuScene / PlayScene / GameOver

04 · MENU SCENE

Concrete scene #1 — the menu

What this page is about: a fully-working scene that paints a title and a 'press space to start' line, then asks the manager to switch to PlayScene when SPACE is hit.

```
class MenuScene(Scene):
    def __init__(self, manager):
        super().__init__(manager)
        self.font = pygame.font.SysFont("Menlo", 48)

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            self.manager.go_to(PlayScene(self.manager))

    def draw(self, screen):
        screen.fill((15, 26, 46))
        title = self.font.render("BRICK BREAKER", True, (255,)*3)
        screen.blit(title, (220, 200))
        # 'press space' line below — omitted for space
```

Two things worth noticing

1. `super().__init__`

Hooks the scene back to its manager so it can call `go_to()` later.

2. `manager.go_to(...)`

MenuScene never says 'I am done' — it just hands the manager a new scene. The next frame, the loop runs PlayScene without touching the game loop code.

05 · SWITCHING SCENES

PlayScene → GameOver · the round trip

What this page is about: how PlayScene decides 'time's up' and hands the manager a GameOver scene. GameOver then loops back to the menu when the player presses R.

```
class PlayScene(Scene):
    def __init__(self, manager):
        super().__init__(manager)
        self.paddle, self.ball = Paddle(), Ball()
        self.lives = 3

    def update(self, dt):
        self.ball.update(dt); self.paddle.update(dt)
        if self.ball.below_screen():
            self.lives -= 1
        if self.lives == 0:
            self.manager.go_to(GameOver(self.manager))
```

```
class GameOver(Scene):
    def handle_event(self, event):
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                self.manager.go_to(MenuScene(self.manager))

    def draw(self, screen):
        screen.fill((0,)*3)
        # 'GAME OVER — press R' text
```

Menu → Play → GameOver → Menu (each arrow is one go_to() call)

Monolithic loop vs scene state machine

Why bother. Read across each row.

DIMENSION	MONOLITHIC LOOP	SCENE STATE MACHINE
Loop body	200+ lines of if-elif-elif	8 lines, never changes
Add a pause screen	Cut up every if branch to also check 'paused'	Write a PauseScene class, manager.go_to(pause)
Reset state	Manually zero every variable	Just construct a fresh scene
Two devs editing	Merge conflicts in the giant loop	Each owns a scene file

 **The scene model is just polymorphism applied to game state**

Each scene is a different subclass with the same interface. The loop only ever calls three methods — it doesn't care which scene answers.

Wire your brick-breaker into a scene state machine

Take Day 11's brick-breaker and retrofit it. Build a Scene base class and a SceneManager. Wrap the existing brick-breaker logic inside a PlayScene. Add a MenuScene (title + 'press space') and a GameOver scene (shows score + 'press R'). The game loop in main() should shrink to under 15 lines.

STRUCTURE

scenes.py → Scene, SceneManager
menu_scene.py → MenuScene
play_scene.py → PlayScene (your
Day 11 logic, refactored)
gameover_scene.py → GameOver
main.py → thin loop only

CLASSES

Scene → handle_event, update,
draw (all pass)
SceneManager → .active,
go_to(scene), and the
three forwarders
MenuScene, PlayScene, GameOver

BEHAVIOR

SPACE on menu → PlayScene
lives == 0 → GameOver
R on game-over → MenuScene
ESC anywhere → quit
main() loop ≤ 15 lines
scenes know nothing of each other

Things that bite when you build this

⚠ Don't switch scenes mid-frame

If PlayScene calls `manager.go_to(GameOver)` inside `update()`, the rest of this frame still runs on PlayScene's draw. Usually fine — but if you delete sprites in `go_to()`, `draw()` crashes. Safer: set a `self.next_scene` flag and let the manager check it after update.

⚠ Each scene reconstructs state

Going Menu → Play → Menu → Play means `PlayScene.__init__` runs twice, with fresh Paddle, Ball, lives. That's the feature — no hidden state leaks between runs. But it also means if you want a 'continue' button, you need to keep the old PlayScene reference around.

💡 on_enter / on_exit hooks

Once the basics work, add optional `on_enter(self)` and `on_exit(self)` methods to `Scene`. Manager calls `on_exit` on the old scene and `on_enter` on the new one during `go_to`. Lets you start music in `MenuScene.on_enter` and stop it on exit.

💡 This is the same pattern as your Day 3 shapes

`Scene/MenuScene/PlayScene` is the same inheritance structure as `Shape/Circle/Box`. Different methods (event/update/draw vs area/perimeter), same idea: one interface, many concrete subclasses, code calls the interface.

Scenes unlock the rest of Week 2

DAY 13 · SAT

Pong (proper)

Two paddles, ball deflection on paddle angle, scoring, basic AI for the right paddle. Today's MenuScene becomes Pong's menu; PlayScene holds the game; GameOver shows the winner.

DAY 14 · SUN

Stretch — Asteroids-lite or polish

Whichever route you pick, the scene wrapper is already in place. Adding LevelSelectScene or a settings menu is a one-file change.

WEEK 4 · TKINTER

Same pattern, different UI

tkinter's frame-swapping pattern is structurally identical: one container, many frames, raise() the active one. Today's mental model carries over.

By Saturday you should be opening Pong with a menu, playing a round, and seeing GameOver — all in one binary.